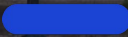


A large group of approximately 50 people, mostly young adults, are posing in a blue boat. They are arranged in several rows, with some sitting on the boat's edge and others standing behind them. Many are smiling and waving. The boat is blue with white text that reads 'xomnia' and 'We make AI work' with a small orange and blue logo. The background is a brick wall with some graffiti.

xomnia

We make   AI work

Harness engineering

Build the control loop around your coding agent

#AGNTCon #MCPCon 2026

A large, solid blue rounded rectangle shape located in the bottom right corner of the slide.

Who am I

- Ji Darwish
- Data Engineer at Xomnia
- Previously Software Engineer
- MSc AI from Utrecht University
- Interest in responsible AI and alignment research



Schedule for today

Everything gets explained first. Then the room goes quiet and you build.

Motivation

why any of this

Guides and sensors

the two axes

The loop

what you will write

Harness

model is not the agent

The problem

one borrowing rule

Build

the room codes

01.

Why?

We already built a calculator

When you learned to code

You built a calculator.

The world already had better ones. Free ones, already on the phone in your pocket.

Today, in this room

You keep the agent you already use.

We are not replacing your coding agent, and we are not competing with it.

You will build a small ring around it out of parts you can take home, and point those parts at rules only your company can write.

Maintainability

architecture rules that execute instead of sitting in a doc

Cost and token budget

stop before the expensive check, cap the repair attempts

Compliance and review

the evidence a human needs to sign off, recorded once

Agents have become a standard

Stripe

1,000+

PRs to production
each week

Shipped by internal coding agents. Humans
review and approve every one.

stripe.com/sessions/2026/developer-keynote

Google

75%

of new code is
AI-generated

Approved by engineers, up from 50% the
previous autumn.

blog.google · Cloud Next 2026 keynote

Uber

70%+

of pull requests
come from agents

Attributed to local or cloud agents across
the engineering org.

uber.com/us/en/blog/efficient-software-factory

Scaling AI at Uber

7x

weekly active users
since Feb

9.4x

weekly agent requests
since Feb

Reported by the companies themselves, not independently audited. Read them as direction of travel, not as benchmarks you are behind on.

the gap this workshop sits in

Delivery is scaling faster than control

Uber's own numbers, and Gartner's forecast.

Happening now

3,600

agent skills built across the software development life cycle at Uber, running more than 30,000 skill executions a day.

Predicted by 2027

40%

of enterprises will reduce or remove autonomous AI agents, due to governance gaps identified only after production incidents occur.

Gartner's point is not that governance is missing. It is that not every agent needs the same controls. An agent that only suggests code can operate with light oversight. An agent that can change or deploy software needs stronger permissions, checks, and approval gates.

Uber, Running a Software Factory Efficiently at Uber Scale. Gartner press release, 26 May 2026: a forecast, not a measurement of your organisation. The claim worth keeping is the ordering, capability first and governance later.

02.

Harness

Model is not the agent

Agent = Model + Harness

Same models, different harnesses

Roughly the same frontier models underneath. The difference you feel is harness.

In your terminal

Claude Code

Codex CLI

Gemini CLI

In your editor

Cursor

Windsurf

Copilot agent mode



CURSOR Windsurf

open code

Every one of them calls the same handful of frontier models. Swap the harness and the model does not change.

Why does it matter?

Resolution rate: the share of task attempts the agent completes successfully.

57.8%

through Terminus 2

52.1%

through Claude Code

Same model both times: Claude Opus 4.5. The only thing that changed is the harness around it.

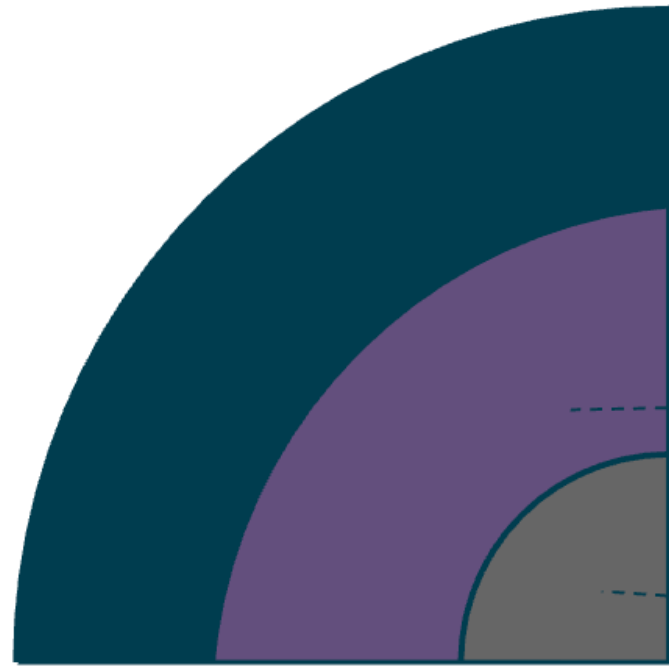
Model	Harness	Resolution rate
GPT-5.2	Codex CLI	62.9%
Claude Opus 4.5	Terminus 2	57.8%
Gemini 3 Pro	Terminus 2	56.9%
GPT-5.2	Terminus 2	54.0%
Claude Opus 4.5	Claude Code	52.1%
Claude Opus 4.5	OpenHands	51.9%

Merrill et al., Terminal-Bench, arXiv 2601.11868, appendix A6. One benchmark, one model, one reported setup. Not a general ranking of harnesses, and not a cost comparison in euros or tokens.

03.

Inner and outer harness

Inner loop, outer loop



User harness

Feedforward and feedback controls put in place by the user of the coding agent for their particular system

Harness built by the builders of the coding agent, e.g. system prompt, code search tools, orchestration, ...

Model

Coding agent or SDK
E.g. Claude Code

<https://martinfowler.com/articles/harness-engineering.html>

The inner loop, conceptually

```
while (true) {  
    response = model.respond(context, tools);  
    if (response.isFinalAnswer()) {  
        return response;  
    }  
    results = tools.execute(response.toolCalls());  
    context.add(response, results);  
}
```

This is the agent you already use. Today it is the part we do not write.

Everything from here on is the ring we draw around it.

Three things this already tells you

The model only chooses.

Every effect comes from a tool the harness allowed.

Context is assembled, not given.

What the model sees is a harness decision.

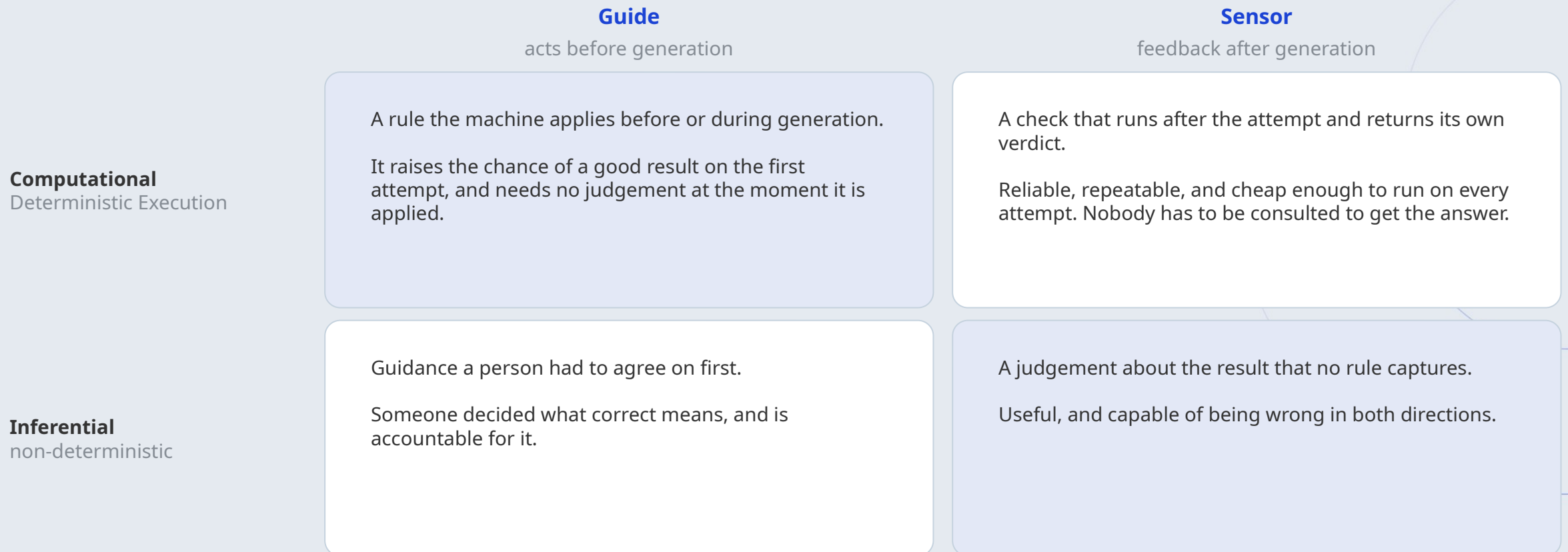
Nothing here decides when to stop.

Stopping is not a model property. It is a rule someone wrote.

Always has been

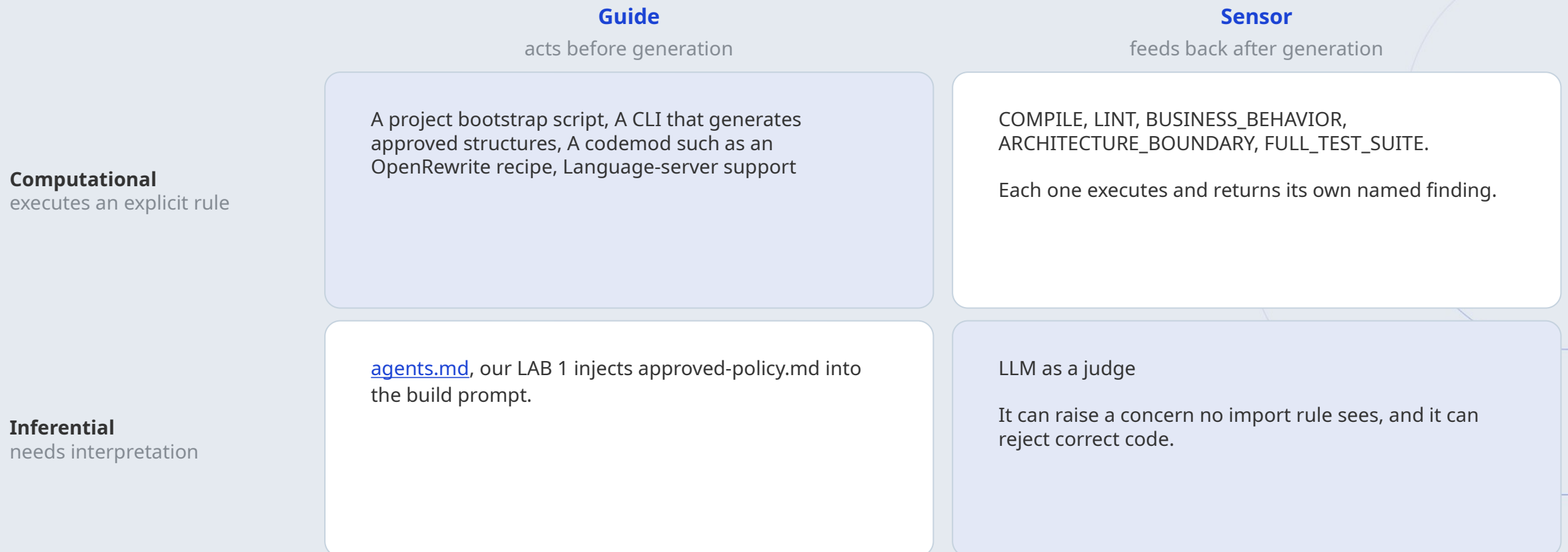
Just loops

When it acts, and how it decides



Guides and sensors, computational and inferential: the terms and the axes are Böckeler's, Harness engineering for coding agent users, martinowler.com.

The same axes, with today's artifacts



Inner loop, outer loop

Outer harness: the code you write today

Inner harness: the coding agent

- reads and edits Java source code
- Has access to tools
- reports success or failure, nothing more

The outer loop owns four things

Policy	the approved rule, sent before the agent acts
Independent checks	named sensors run after each attempt, outside the agent's reach
Repair budget	how many attempts a failure is worth, and what stops the loop
Acceptance	what evidence, on which source version, permits a yes

Enforced, not promised

04.

The problem you will work on

the codebase you will work in

The Bookshelf

One physical copy of each book. Nine classes. Small on purpose.

```
bookshelf/  
  approved-policy.md  
src/main/java/workshop/bookshelf/  
  domain/  
    Book, Member, Loan  
    BorrowOutcome  
  service/  
    BorrowService  
  storage/  
    InMemoryBookshelf  
src/test/java/workshop/bookshelf/  
  BorrowServiceTest  
  ArchitectureTest  
  BorrowPolicyTest
```

At work the business rule is pricing, permissions, time zones or idempotency. The size of the example is not the point.

Who holds the copy?

1

Alice borrows book 1

active borrower: Alice



2

Bob requests book 1

Bob's result: ?
active borrower: ?



3

Alice returns book 1

active borrower: nobody



4

Bob borrows book 1

Bob's result: ?
active borrower: ?

Who holds the copy?

1

Alice borrows book 1

active borrower: **Alice**

2

Bob requests book 1

Bob's result:
BOOK_UNAVAILABLE
active borrower: **Alice**

3

Alice returns book 1

active borrower: **nobody**

4

Bob borrows book 1

Bob's result: **BORROWED**
active borrower: **Bob**

run this before you write anything

A green suite that asserts nothing

bookshelf/.../service/BorrowService.java

```
public BorrowOutcome borrow(long bookId, long memberId) {
    if (!shelf.hasBook(bookId)) return BOOK_NOT_FOUND;
    if (!shelf.hasMember(memberId)) return MEMBER_NOT_FOUND;

    // the agreed second-borrow rule is not checked here
    shelf.record(new Loan(bookId, memberId));
    return BORROWED;
}
```

bookshelf/src/test/java/.../BorrowPolicyTest.java

```
@Test
void aSecondMemberCannotBorrowAnAlreadyLoanedBook() {
    var service = new BorrowService(shelf());
    service.borrow(1, 10);

    // no assertion follows
}
```

```
./mvnw -pl bookshelf test
```

PASS

and Bob can still replace Alice as the active borrower.

Feature: Login

As a user

I want to be able to log in to the application

So that I can access my account information

Scenario: Successful login

Given I am on the login page

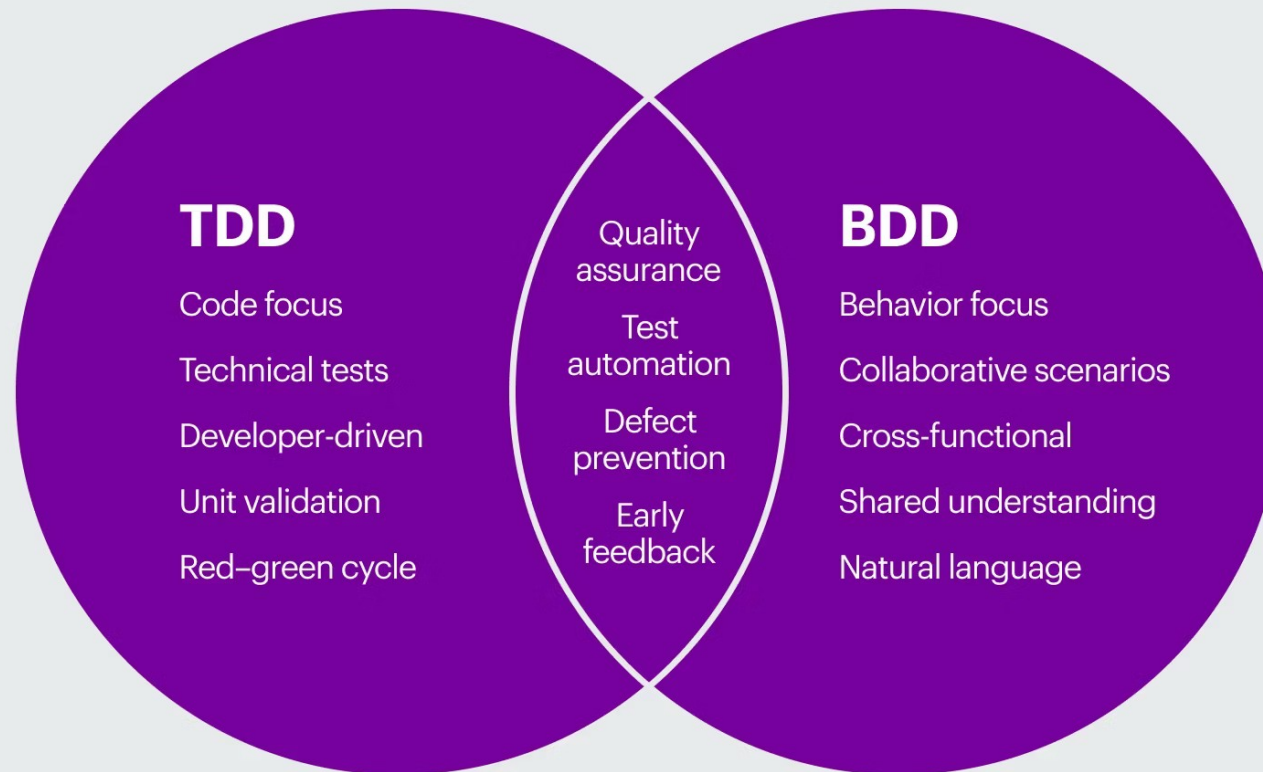
When I enter my username and password

And I click the login button

Then I should be taken to the dashboard page

TDD & BDD

TDD vs. BDD



<https://www.ramotion.com/blog/tdd-vs-bdd/>



05.

The loop you will build

Looping is not reliability

%82

correct after one revision

%67,3

correct after two

%84,7

ever correct at some point

Stale evidence is the mechanism

Verification traces from an earlier version harmed 34 of 135 initially correct attempts. With current traces: 4 of 135.

What you already built, for this reason

One repair at most (for now). Acceptance judged on the final source attempt. Never reuse a PASS from before a repair.

A repair budget is not stinginess. It is the control that stops a loop thrashing away a correct answer.

Gao, Yang and Yang, Looping Is Not Reliability, arXiv 2607.24604.

Three decisions, **three labs**

Each lab is one decision you make and can defend.

LAB 1

the guide,
then the sequence

The approved rule reaches the agent,
and the checks run in an order you
chose.

LAB 2

bound the repair,
and make it useful

One repair at most, carrying every
failing finding in a single prompt.

LAB 3

decide acceptance
on the final attempt

A PASS from before the repair does
not count toward the answer.

The sequence *you* wire

build with approved policy

COMPILE

prerequisite: a FAIL skips everything below

LINT

BUSINESS_BEHAVIOR

ARCHITECTURE_BOUNDARY

independent findings, collected for one repair prompt

FULL_TEST_SUITE

only if all three focused checks PASS

one repair for current application FAIL findings, or stop

repeat the entire sequence on the repaired source

accept the exact final PASS set, or report UNRESOLVED

Two rules that are not negotiable

- An ordinary FAIL must not hide another independent finding.
- An ERROR stops the sequence. It is not a result to repair against.

./harness.sh check

```
PASS  COMPILE
PASS  LINT
FAIL  BUSINESS_BEHAVIOR
PASS  ARCHITECTURE_BOUNDARY
SKIPPED FULL_TEST_SUITE
```

Acceptance is decided on **the final attempt**

Attempt 1

BUSINESS_BEHAVIOR

FAIL

ARCHITECTURE_BOUNDARY

PASS

FULL_TEST_SUITE

SKIPPED

Attempt 2, after one repair

BUSINESS_BEHAVIOR

PASS

ARCHITECTURE_BOUNDARY

FAIL

FULL_TEST_SUITE

SKIPPED

Verdict: UNRESOLVED

Two green labels exist. They belong to two different versions of the source, and neither version passes everything that was required. Accumulating green across attempts is the single easiest way to build a loop that accepts broken work.

The outer harness is getting attention

No single tool has settled the whole design. The same pieces keep appearing separately.

Instructions

Claude code, codex...

repository-local guidance that reaches the agent first

Deterministic hooks

Claude Code, GitHub Copilot...

commands at lifecycle events: format, test, scan, block

Checks and repair

Dagger, Aider, Qodo Cover

run checks after edits, feed failures back, cap iterations

External verification

Harbor

Evaluate agent's result with task-defined verifiers outside the agent loop

Guarded delivery

GitHub Agentic Workflows

sandboxed jobs, read-only defaults, timeouts, approval gates

Many pieces exist. Deciding which properties matter, which sensors establish them, what justifies another attempt, and who may accept the result is still yours.

06.

Over to you

Contact

Thank you for participating in this workshop



Data Engineer

Ji Darwish

ji.darwish@xomnia.com

github.com/JiDarwish/outer-harness-workshop



Xomnia B.V.
Amsterdam
xomnia.com



xomnida



We make AI work

visit us at:
xomnia.com